

Exercise #5 – Extended Lab Proposal and Exploratory Coding

Mason McGaffin

Exercise #5

ECEN 5763 Embedded Computer Vision

University of Colorado Boulder

July 15, 2026

Contents

1 THE STANLEY VEHICLE.....	3
2 EXPLORATORY PROGRAMMING.....	4
3 PROJECT REFINEMENT	11
4 PROJECT PROPOSAL.....	12
5 REFERENCES.....	14
6 APPENDIX	15

1 The Stanley Vehicle

Read Sebastian Thrun's design paper on the Stanley vehicle which won the DARPA challenge – "Stanley: The Robot that Won the DARPA Grand Challenge", Journal of Field Robotics 23(9), 661–692 (2006). Describe in good detail how machine vision worked for Stanley and why it was necessary. Was the LIDAR used by Stanley necessary? Could it have instead used a more sophisticated machine vision system with multiple cameras?

The Stanley vehicle was a groundbreaking autonomous vehicle developed by Stanford University that navigated 132 miles of rugged terrain entirely autonomously by using a combination of LIDAR and machine vision.

The machine vision system in Stanley consisted of a single roof mounted camera. With only one camera, the system could not measure depth, distance, or geometric height/width accurately. The team, led by Thrun, used LIDAR rangefinders and developed an adaptive computer vision algorithm that utilized machine learning to overcome these challenges. The system operated in a continuous loop, processing frames at ~12Hz. The loop used LIDAR data to identify safe areas to drive within 25 meters. These areas were projected onto the 2D frame captured by the camera. Then, the pixels corresponding to the safe zone were used to train a machine learning model using positive reinforcement. The trained classifier was then applied to the rest of the camera image to search for drivable regions beyond the LIDAR range. The rugged terrain the Stanley vehicle traversed changed consistently which raised the need for continuous adaptation. The algorithm used a high learning rate to adapt to change quickly.

Machine vision was necessary due to range limits of LIDAR. With only around 25 meters of range, the top speed and navigation ability of the vehicle was significantly inhibited. Machine vision allowed the vehicle to travel faster and also safer. If, for example, terrain further out was classified as difficult, the vehicle could slow down to allow for proper analysis using LIDAR.

Lidar was very necessary for the system because only one camera was used. The camera can only evaluate appearance whereas LIDAR provides the ability to evaluate physical distance and geometry. There are many factors such as lighting, shadows, color, etc. that can change the appearance of a scene. The LIDAR system was necessary to mitigate the noise and provide a ground truth for immediate surroundings.

It is significant that the system only consisted of one forward-facing camera. If more than one camera was used, with careful configuration and calibration, then the LIDAR could possibly be eliminated. However, at the time (2005), this was unviable due to hardware and algorithmic constraints. The introduction of more cameras and the algorithms used to process the data would have contributed significant latency to the system, which most likely would have been unviable. Even today, multi-camera systems are still highly sensitive to rapidly changing environments, such as the desert that the Stanley vehicle crossed. Shadows, color, etc could have caused major issues that the LIDAR system would correct using its physical data.

2 Exploratory Programming

Research, write code and demonstrate processing of an application (creative or AV/ADAS road test) video to support a driving assistant OR creative project. Part of your work for the AV/ADAS or creative project is to evaluate and choose algorithms from OpenCV or other libraries (e.g., CUDNN) to use in your design OR implement your own algorithms bottom-up.

For a creative project of your own interest create an exploratory program to assess feasibility of your application as follows:

- a) Select one or more key algorithms and verify that there is an OpenCV implementation you can re-use or that you can implement from the ground up and test on a single or set of still images (e.g., stitch together two or more images using feature point correspondence).*
- b) Modify your example above to work on a continuous stream of images from pre-recorded video that you capture which represents a challenge set for your project of interest (e.g., stitches together a panorama taken from two or more cameras as continuous video).*
- c) Provide processed video and input video to show that your exploratory prototype works or explain why it does not.*

My final project follows the creative project guidelines. It will implement a real-time, multi-threaded facial recognition system designed to process live webcam feeds or recorded video using OpenCV with CUDA acceleration for GPU offloading. The core pipeline utilizes a CUDA-accelerated Haar Cascade algorithm for face detection, followed by a Local Binary Patterns (LBP) algorithm to identify individuals against a custom database. Additionally, the project includes a stretch goal to replace the traditional LBP method with a more robust machine learning model for facial recognition.

For the exploratory aspect, I first chose to explore the Haar Cascade algorithm. This algorithm is used for facial detection. It loads a detection model that is formatted in an xml file. Once the model is loaded, the algorithm detects Haar features using simple rectangular filters and eliminates regions of the image that do not contain a face using multiple classifiers. OpenCV provides an API for a CUDA accelerated detection functions. In order to compare performance, I used syslog tracing and the `getTickCount()` API. I was able to average and visualize the detection time across 200 frames, as seen below.

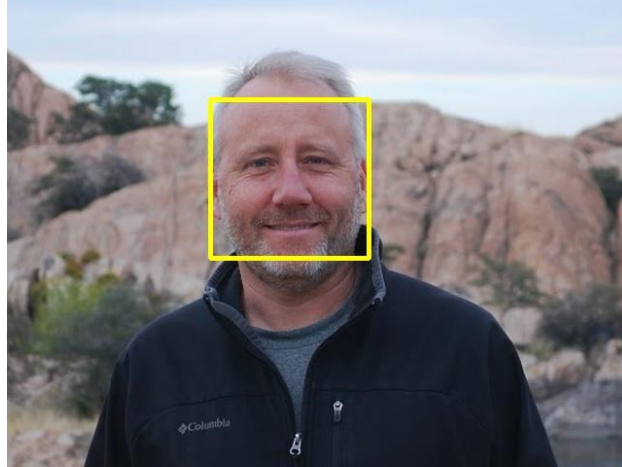


Figure 2.1 simple_detect output

```
$ python3 avg_time.py simple_detect_syslog.txt
simple_detect_syslog.txt

Successfully parsed 200 frames.
=====
AVERAGE PROCESSING TIME
=====
Minimum time : 30.08
Maximum time : 105.64
Average time : 91.40
=====
Plot successfully generated and saved to 'simple_detect_detection_times.png'
```

Figure 2.2 simple_detect syslog processing output

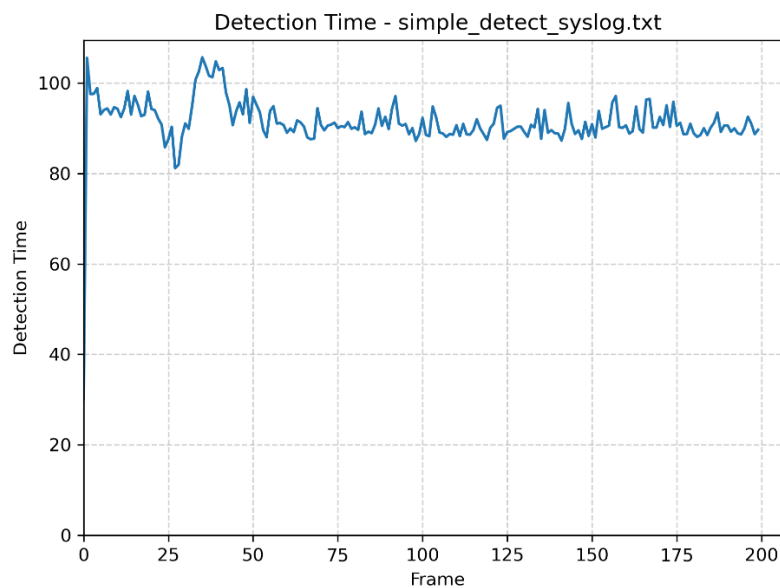


Figure 2.3 simple_detect detection time chart

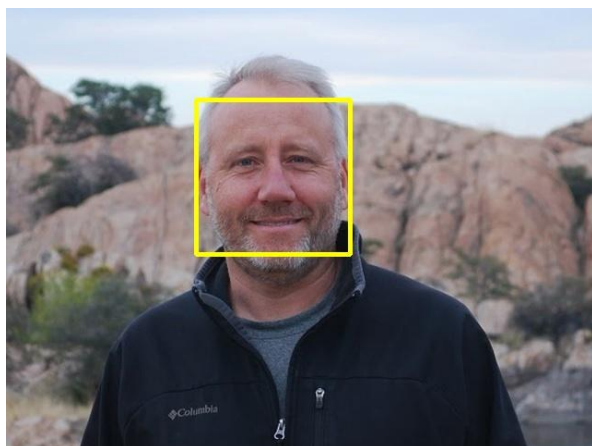


Figure 2.4 cuda_detect output

```
$ python3 avg_time.py cuda_detect_syslog.txt
cuda_detect_syslog.txt

Successfully parsed 200 frames.
=====
AVERAGE PROCESSING TIME
=====
Minimum time : 26.13
Maximum time : 95.91
Average time : 46.34
=====
Plot successfully generated and saved to 'cuda_detect_detection_times.png'
```

Figure 2.4 cuda_detect syslog processing output

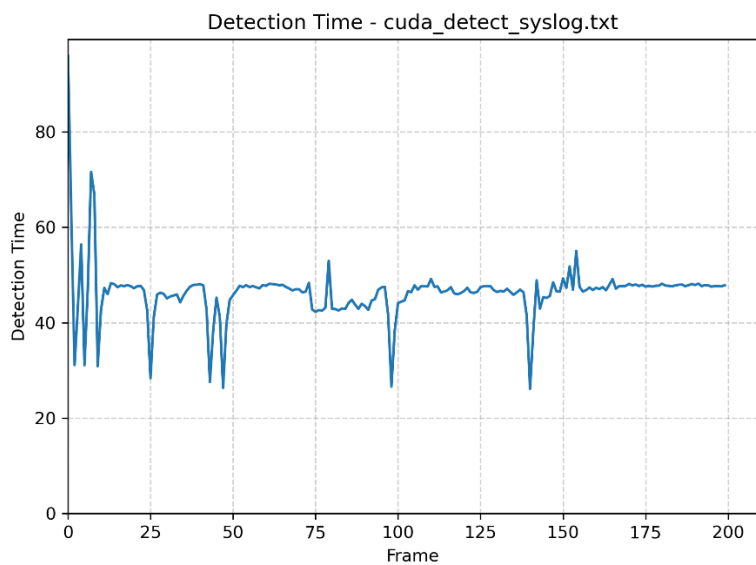


Figure 2.5 cuda_detect detection time chart

As seen when comparing Figure 2.2 and Figure 2.4, the CUDA accelerated detection reduces detection time by nearly half. However, Figure 2.5 shows that there is higher variability in the detection time seen by the larger spikes in the graph.

Next, I moved on to explore a basic recognition algorithm using a local binary pattern (LBP) classifier. OpenCV offers a LBPHFaceRecognizer class from the face module. This module trains on labeled images. For simplicity, the exploratory example only used one image as the training image. First, I demonstrated the ability to detect the face using the standard Haar Cascade classifier and crop the image to just the face as shown in Figure 2.6.

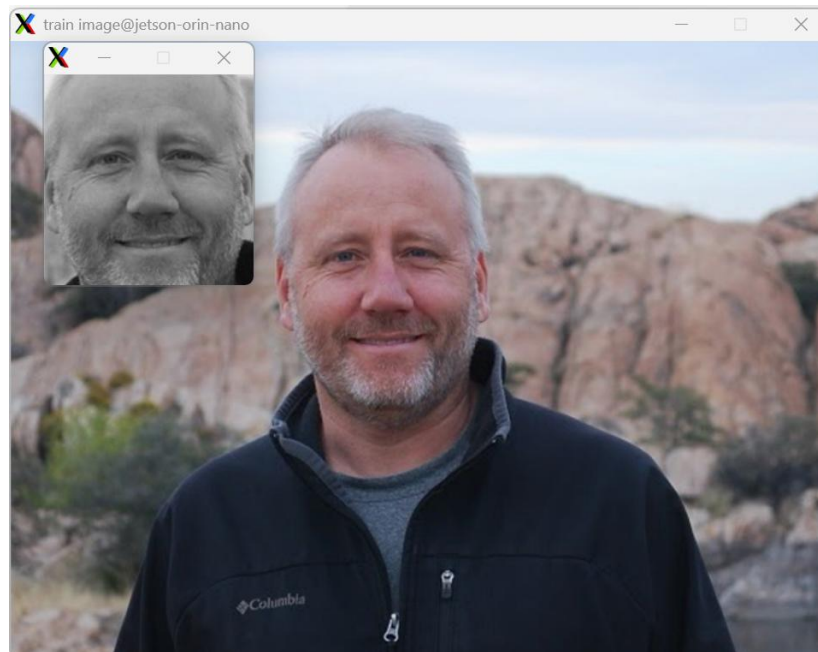


Figure 2.6 Detecting and cropping face

Once, I demonstrated this, I implemented the recognition methods. The LBP classifier works by labeling each pixel of an image through thresholding its surrounding neighborhood and converting the result into a binary structure. For each face detected, the application tries to label it against the trained model and provides a confidence/distance value that indicates how strongly of a match the detected face is (closer to 0 is better match).

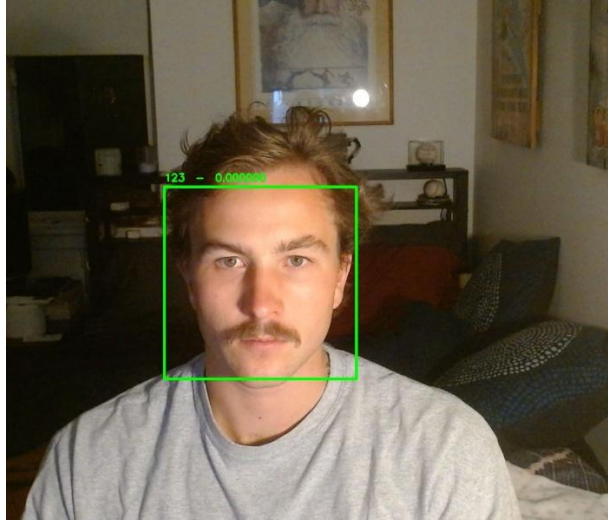


Figure 2.7 Facial recognition with identical training and test image

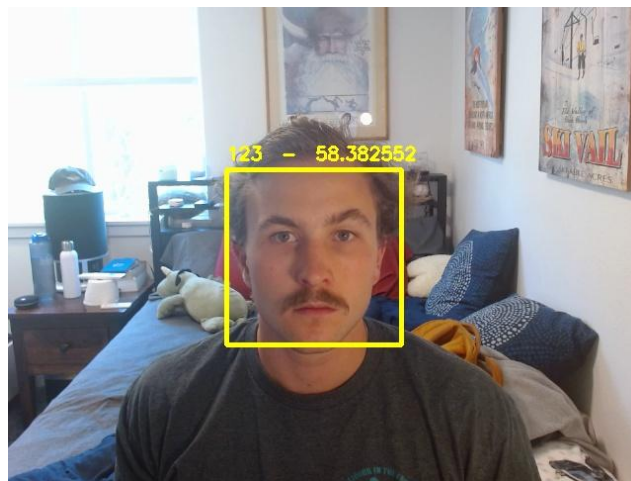


Figure 2.8 Facial recognition with same individual

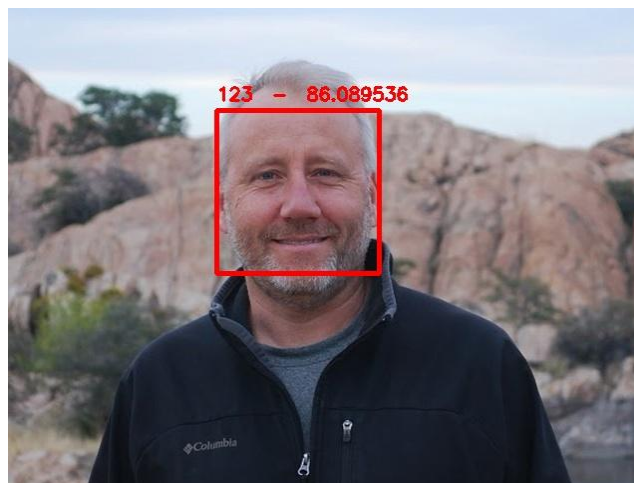


Figure 2.8 Facial recognition with different individual

As seen in the figures above, the application correctly identifies the faces in the frame. Because there is only one individual (one image) that the model was trained on, every face is compared to that image. The exact image (Figure 2.7) shows zero uncertainty. The images are identical, so the face detected is a perfect match. We also see that the model shows greater confidence (closer to 0) when the test image contains the individual the model was trained on, as expected. It should be noted that the confidence is not as high as we hoped because we only trained the model on one image and the lighting changed between training and reference. I measured the total recognition time (detection + recognition) and plotted the confidence of each face recognized.

```
mason@jetson-orin-nano:~/ECV_5763/ex5 (ECV_5763::ex5)
$ python3 avg_lbp.py lbp_syslog.txt

Successfully parsed 205 frames.

=====
AVERAGE RECOGNITION TIME
=====
Minimum time : 86.27
Maximum time : 116.13
Average time : 93.97
=====
Minimum confidence : 58.3826
Maximum confidence : 98.5548
=====
Plot successfully generated and saved to 'lbp_recognition_times.png'
Plot successfully generated and saved to 'lbp_confidences.png'
mason@jetson-orin-nano:~/ECV_5763/ex5 (ECV_5763::ex5)
```

Figure 2.9 lbp_recognize recognition time

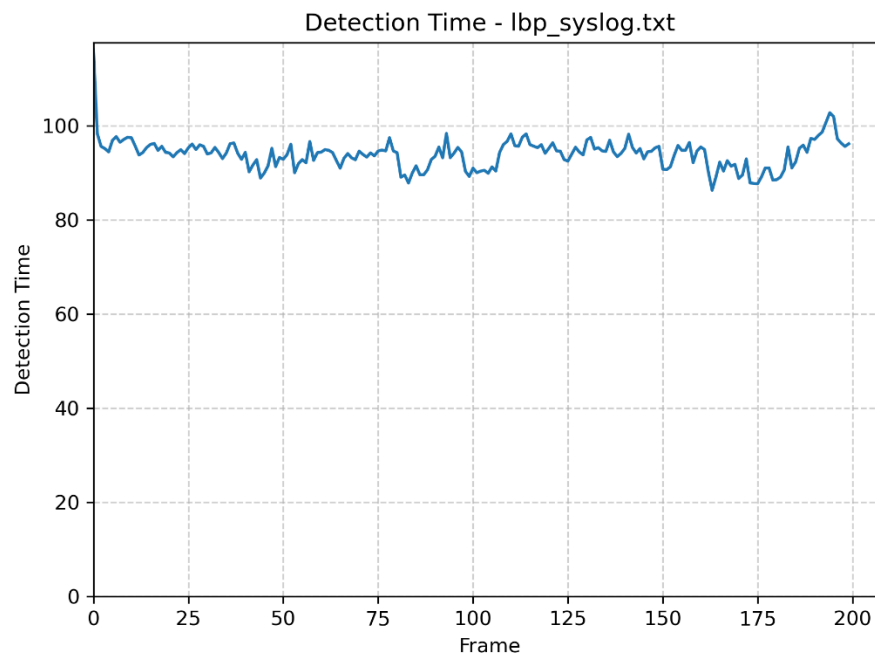


Figure 2.10 lbp_recognize recognition time

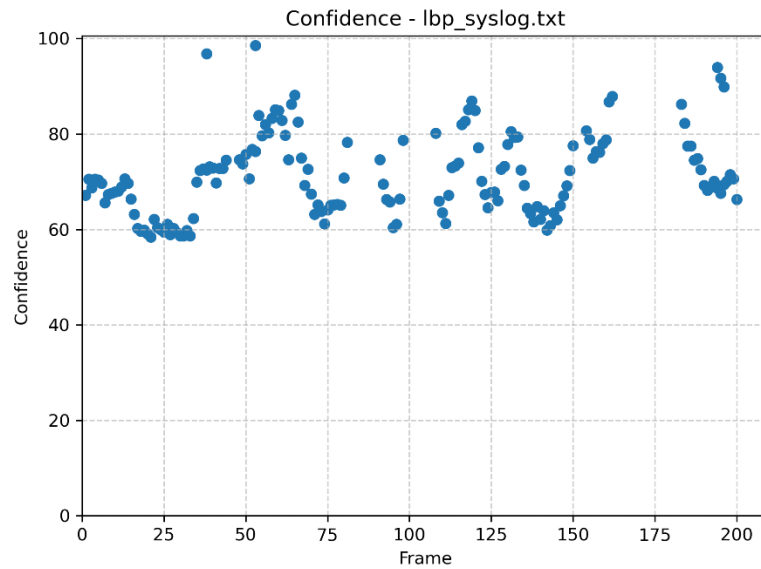


Figure 2.11 lbp_recognize confidence

The confidences were plotted as a dot plot as opposed to a normal line plot because there were instances when an image in the background was identified as a face and provided two confidence values for a single frame. There were also frames where no face was detected. This can be observed in lbp_output.mp4.

3 Project Refinement

Read texts, lecture notes, and supporting materials, consider example code, and choose an Applied Computer Vision application and improvements to your exploratory work to further refine for demonstration. Explain how you will improve in good detail.

- a) How do you plan to improve your features for final test and demonstration? Higher frame rates? Better detector performance? More testing? Other?*
- b) Why are your features relevant to the course?*
- c) What will you learn by developing these features?*

There were many aspects of the exploratory applications, discussed in Section 2, that need improvement.

First, I was disappointed in the degree of speedup from using the CUDA accelerated OpenCV functions. This emphasizes the need for a multi-threaded approach. I will implement Pthreads and also explore OpenMP if needed (and time permits) to meet Stretch performance requirements.

The facial detection model currently uses the provided xml file for front facing persons. This model works well for static images directly facing the camera. However, it was observed to have difficulty with individuals moving laterally and rotating their head. I will research online forums and potentially AI sources to improve the xml model file.

For the facial recognition model, I will expand the training data by creating an image database of eight individuals. Each individual will have eight labelled headshots used to train the recognition model. A cutoff confidence level will be chosen, from testing analysis, to determine if a face does not match any individuals in the database. My stretch goal for facial recognition is to implement a ML model (YOLO) to improve facial recognition confidence. This option will be pursued after meeting all objectives for standard LBP recognition.

My features for this project are facial recognition and CUDA acceleration. To deliver results for these features I have outlined Minimum, Target, and Optimal objectives for the features as seen in Table 4.1. By developing these features and targeting performance and reliability objectives, I will gain knowledge and experience of computer vision algorithms, OpenCV functions, CUDA acceleration, POSIX multi-threading and synchronization, timing and performance tracing and analysis, and ML algorithms.

4 Project Proposal

Write a 3-paragraph proposal for your application that describes it in detail with block diagrams if needed or other figures so that it is clear what you will implement and demo for Lab #6. Define your project goals with at least three features at each of the following levels of challenge:

- a) Optimal – this should include advanced processing, I/O and/or storage optimization to increase throughput and any real-time or interactive requirements you have as well as performance related to accuracy. Be specific in terms of objective metrics that are ideal (frames/sec, TP/FP/TN/FN, by feature) for real-world use.*
- b) Target – goals should include the key features of your design (e.g., frame rates, detection, tracking, recognition, depth estimation, transformation from image to point cloud, ...) and expected performance in terms of frames/sec, detection performance, and use of CPU cores and/or GP-GPU required to show feasibility. You can also add additional features here so that # of features > # of team members.*
- c) Minimum – simple objectives that define basic success (e.g., continuous skeletonization of a hand and recognition of a specific set of gestures, or at least one or more pedestrians detected, lane lines found, or similar) to show potential. You can again add additional features here so that # of features > # of team members + 1.*

Facial Recognition

This individual project will implement a real-time facial recognition system capable of detecting human faces and identifying corresponding individuals. The project will accept both live webcam feed and recorded video as input test cases. The system will continuously process video frames using OpenCV with CUDA acceleration to offload computational work to the GPU.

The application will be implemented as a multi-threaded OpenCV pipeline consisting of threads dedicated to video ingesting, GPU frame processing, and output rendering. The user will have the ability to configure a live GUI interface or recorded video output. The application will measure performance in frames per second, and facial recognition accuracy and reliability will be evaluated using manual inspection in different environmental conditions.

Initial face detection will be performed using a CUDA-accelerated Haar Cascade algorithm, isolating face objects with the frame and marking them with bounding rectangles. Once a face is detected, a Local Binary Patterns (LBP) algorithm will analyze facial features to identify the individual against a custom database (ATT updated with myself and roommates' images) and display the individual's name on the output image.

As a stretch goal for this project, the traditional LBP facial recognition may be replaced with a YOLO machine learning model.

Objective	Feature 1 – CUDA acceleration	Feature 2 – Facial recognition	Throughput	Reliability
Minimum	GPU acceleration does not work but CPU functionality still works	Faces are detected but individuals are only sometimes identified in ideal environments 2+ faces recognized	7+ frames/sec	$P = TP / (TP + FP) > 0.7$ $R = TP / (TP + FN) > 0.3$
Target	GPU acceleration functions are used but they do not significantly improve performance.	Facial recognition using LBP is accurately identifies individuals in most environments. 4+ faces recognized	15+ frames/sec	$P = TP / (TP + FP) > 0.8$ $R = TP / (TP + FN) > 0.4$
Optimal	GPU acceleration significantly improves system performance.	Facial recognition using LBP is robust and accurately identifies individuals in cluttered and light-sensitive environments. 8+ faces recognized ML model accurately identifies individuals in some environments.	24+ frames/sec	$P = TP / (TP + FP) > 0.9$ $R = TP / (TP + FN) > 0.5$

Table 4.1 Objective Definitions

5 References

- [1] S. Thrun *et al.*, "Stanley: The robot that won the DARPA Grand Challenge," *J. Field Robot.*, vol. 23, no. 9, pp. 661–692, Sep. 2006, doi: 10.1002/rob.20147.
- [2] JetsonHacks, "OpenCV 4 + CUDA on Jetson Nano," *JetsonHacks*, Nov. 22, 2019. [Online]. Available: <https://jetsonhacks.com/2019/11/22/opencv-4-cuda-on-jetson-nano/>
- [3] S. Siewert, "Tutorial: CUDA and Multi-Core," *ECV-5763-CU Lecture Documents*, ESEE Web Resources, University of Colorado Boulder. [Online]. Available: <https://o365coloradoedu-my.sharepoint.com/>
- [4] OpenCV, "OpenCV 4.10.0 Documentation." 2024 [Online]. Available: <https://docs.opencv.org/4.10.0/>

6 Appendix

GitHub repository: https://github.com/mamc3334/ECV_5763/tree/ex5

Pull request: https://github.com/mamc3334/ECV_5763/pull/6

Forked ECV GitHub repository: <https://github.com/mamc3334/ECV-ECEE-5763-mcgaffin>

Pull request: N/A